PROBLEM STATEMENT / PROJECT VISION

Upside Down

One idea. A coordinated team. Real understanding.

The problem

Hackathon students often work across several laptops using different AI coding tools. Isolated conversations lead to repeated context, unclear ownership, incompatible frontend and backend code, and difficult integration. Students may also struggle to explain the code their tools produced.

The challenge

Design a shared workspace that turns a problem statement into a clear architecture and individual laptop assignments, keeps contributors aligned during development, and helps students understand and present the resulting project.

The primary pipeline

01 / Upload the problem
Bring a PDF, Markdown document or written brief into the workspace.

02 / Agree on the idea and architecture
Refine the idea, define modules and freeze a common integration contract.

03 / Give every laptop its role
Share common context, architecture and contract; give each laptop its own task prompt.

04 / Build, review and integrate
Track parallel work, communicate changes, review patches and resolve conflicts before integration.

05 / Understand and present
Explore a visual workflow, learn the concepts and explain the project confidently.

Reference demo: AgentSync

The current Upside Down walkthrough uses one consistent example: AgentSync, a local-first workspace where coding agents share tasks, detect contract changes, flag conflicting edits and queue cloud updates when internet access is unavailable.

Four laptops, clear ownership

Laptop A / P1 - Backend and coordinator
Own task coordination, event persistence, SQLite and conflict rules.

Laptop B / P2 - Frontend and experience
Own the dashboard, activity views, task screens and user interactions.

Laptop C / P3 - API and realtime
Own HTTP routes, validation, shared schemas and event streaming.

Laptop D / P4 - Bridge and quality checks
Own agent event scripts, optional GitHub sync and the integration scenario.

The learning feature

An interactive map explains how an action travels from the frontend through the API and backend, then returns to the interface. Students explore module responsibilities, illustrative files and functions, concepts such as persistence and idempotency, and a short presentation narrative.

What this prototype demonstrates

File selection followed by a 25-second loading reveal; a fixed architecture map; four role-specific prompt packs with shared documents; sample tasks, agents, messages, reviews, conflicts and queue interactions; and a visual teaching walkthrough.

The current demo uses predefined content and simulated activity. It does not analyze the uploaded PDF, invoke an AI model, connect real coding agents, execute their work or integrate a generated application. Those capabilities belong to the intended product roadmap.

Desired outcome

A hackathon team can see what to build, who owns each part, how the parts fit together, and how to explain the project. This brief is suitable for uploading into the existing fixed-reference demonstration.